ITA0605 – MACHINE LEARNING

ASSIGNMENT 2 – CO2

Neural Network Representation, Perceptron Learning, Backpropagation, Deep Networks, Regularization and Optimization

Submission details	Information
Student Name	Thikkavarapu Sanhith
Register Number	192311228
Programme / Department	B-Tech/CSE
Course	ITA0605 – Machine Learning
Assignment	Assignment 2 – CO2

Q1 – DESIGN A NEURAL NETWORK REPRESENTATION FOR A CLASSIFICATION PROBLEM (6 MARKS)

1.1 Problem formulation and input representation

The manufacturing task is binary classification. Each component is described by three numerical measurements and must be assigned one of two labels: Acceptable (0) or Defective (1). The input vector is:

$x = [x1, x2, x3]^T$ = [Surface Roughness, Dimensional Deviation, Vibration Level]^T

Feature	Numerical type	Representation	Reason for relevance
Surface roughness	Continuous numeric	Measured physical value; standardised before training	Surface texture can indicate wear, poor finishing or material defects.
Dimensional deviation	Continuous numeric	Measured deviation from nominal dimension; standardised	Large deviation may indicate machining or assembly error.
Vibration level	Continuous numeric	Measured vibration magnitude; standardised	Abnormal vibration can indicate imbalance, looseness or tool damage.
Target	Binary categorical	Acceptable = 0; Defective = 1	Supervised label for model training.

Preprocessing is necessary because the three measurements may use different units and ranges. Standardisation transforms each feature using statistics fitted on the training set only:

$x_scaled,j = (xj - \mu j,train) / \sigma j,train$

Missing sensor readings should be imputed with a training-set median (or a documented sensor-specific rule). Outliers should be investigated against physical operating limits rather than

automatically deleted. All learned preprocessing values must be reused unchanged for validation, test and production observations. The target is not standardised.

1.2 Candidate architecture A: no hidden layer

The first architecture is a logistic-regression model represented as a single perceptron with a sigmoid output. It has three input units and one output unit. The output probability estimates the probability that the component is defective.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$p(\text{Defective} \mid x) = \sigma(z) = 1 / (1 + e^{(-z)})$$

Decision rule: predict Defective (1) when $p \geq 0.50$; otherwise predict Acceptable (0). The threshold can later be adjusted using validation data if the costs of missed defects and unnecessary inspections are unequal.

Trainable parameters = 3 weights + 1 bias = 4

1.3 Candidate architecture B: one hidden layer

The second architecture has three input units, four ReLU hidden units and one sigmoid output unit. The hidden layer can learn nonlinear combinations of roughness, dimensional deviation and vibration. It is therefore a better candidate when defect formation depends on interactions among the measurements.

$$h = \text{ReLU}(xW_1 + b_1)$$

$$p = \sigma(hW_2 + b_2)$$

$$\text{ReLU}(a) = \max(0, a)$$

Trainable parameters = $(3 \times 4 + 4) + (4 \times 1 + 1) = 16 + 5 = 21$

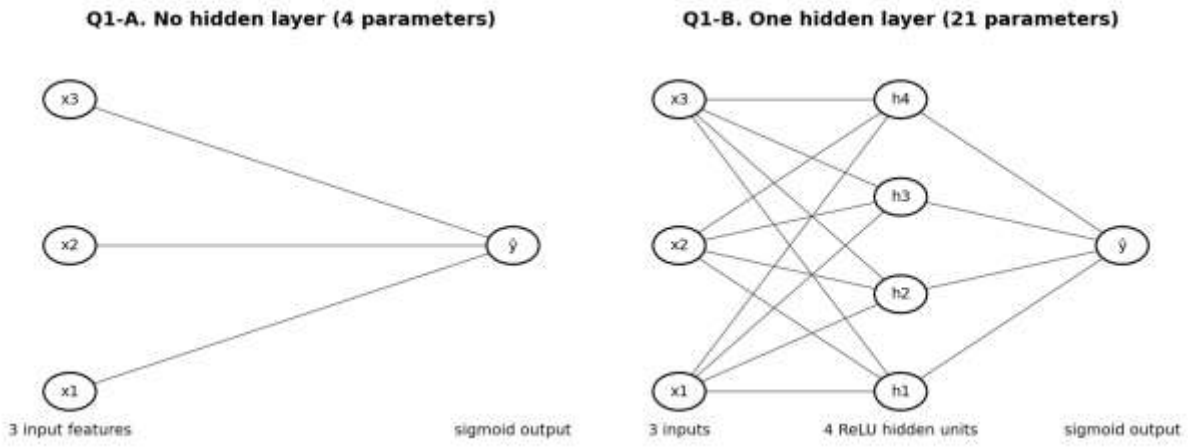


Figure 1. Two candidate neural-network representations for the three sensor features.

1.4 Worked forward pass for architecture A

For one illustrative standardised input, assign x = [0.5, −0.2, 1.0], weights w = [0.4, −0.3, 0.2] and bias b = −0.1. These are example values chosen to demonstrate the calculation; they are not fitted manufacturing parameters.

$$z = (0.5)(0.4) + (-0.2)(-0.3) + (1.0)(0.2) - 0.1$$

$$z = 0.20 + 0.06 + 0.20 - 0.10 = 0.36$$

$$p = \sigma(0.36) = 1 / (1 + e^{(-0.36)}) = 0.5890$$

Since $0.5890 \geq 0.50$, the model predicts Defective (class 1). A production system should calibrate its threshold with held-out data and the relative cost of a missed defect.

1.5 Architecture recommendation and evidence needed

The hidden-layer architecture is the preferred design hypothesis because the inspection team expects interactions among the three measurements. A component might be defective only when vibration is high together with a dimensional deviation, even if neither value alone crosses a simple threshold. A single sigmoid output over the raw features can only form a linear decision boundary in the standardised input space. A hidden layer with nonlinear activation can form a more flexible boundary.

The hidden layer should not be selected solely because it is more complex. The team should compare both candidates using the same training, validation and test partitions; repeated random seeds; validation loss; precision and recall for the Defective class; confusion matrices; calibration; and the

operational cost of false negatives. An ablation showing that the hidden layer improves validation performance consistently would justify its additional parameters and computation.

Q2 – INVESTIGATE THE LEARNING BEHAVIOUR AND LIMITATIONS OF A PERCEPTRON (4 MARKS)

2.1 Complete truth tables

Let x1 indicate Packaging Complete and x2 indicate Quality Check Passed. Dispatch is allowed only if both indicators are 1, which is the AND function. The changed review flag is 1 when exactly one indicator is 1, which is XOR.

x1	x2	Dispatch AND	Review flag XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

2.2 Can a perceptron learn the original dispatch rule?

Yes. The AND function is linearly separable. The four input points are the corners of a square. The three negative examples lie at (0,0), (0,1) and (1,0), while the only positive example is (1,1). A single line can separate the positive corner from the other three points.

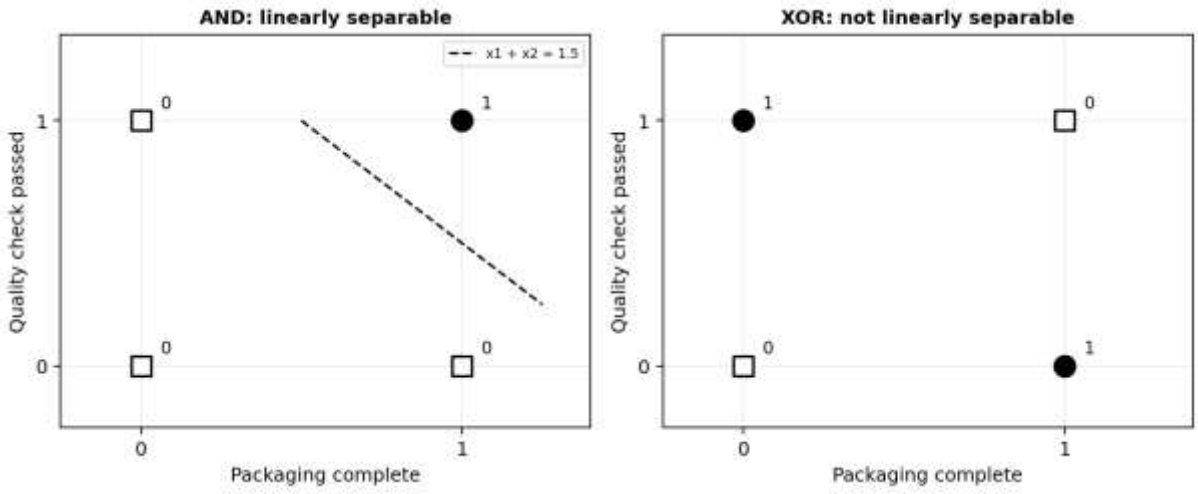


Figure 2. Geometric view of AND and XOR. A single straight line separates AND, but no single line separates the XOR labels.

2.3 Perceptron configuration and first-epoch update trace

The perceptron uses initial weights w = [0, 0], initial bias b = 0, learning rate η = 0.10, threshold = 0.50 and maximum 20 epochs. Prediction is 1 when the net input is at least 0.50; otherwise it is 0. The online update rule is:

$\hat{y} = 1$ if $(w^T x + b) \geq 0.50$, otherwise $\hat{y} = 0$

$e = \text{target} - \hat{y}$; $w \leftarrow w + \eta\, e\, x$; $b \leftarrow b + \eta\, e$

Before training, all four patterns are predicted as 0 because the initial net input is 0. During epoch 1, the first three examples are correctly predicted as 0. The final row (1,1) is misclassified and causes one update:

$x = [1,1]$, target = 1, $\hat{y} = 0$, $e = 1$

$w_{new} = [0,0] + 0.10 \times 1 \times [1,1] = [0.10,0.10]$

$b_{new} = 0 + 0.10 \times 1 = 0.10$

Epoch	Misclassifications	Weights after epoch	Bias after epoch	Predictions in row order
1	1	[0.1, 0.1]	0.10	[0, 0, 0, 0]
2	1	[0.2, 0.2]	0.20	[0, 0, 0, 1]
3	0	[0.2, 0.2]	0.20	[0, 0, 0, 1]

At epoch 2, the point (1,1) still has net input 0.10 + 0.10 + 0.10 = 0.30, below the 0.50 threshold. It is updated again to w = [0.20,0.20], b = 0.20. The net input for (1,1) becomes 0.60, so it is classified as 1. Epoch 3 has zero mistakes and training stops.

Final weights	Final bias	Final predictions	Converged?
[0.2, 0.2]	0.20	[0, 0, 0, 1]	Yes, after 3 epochs

2.4 Changed requirement: XOR at two learning rates

XOR requires the output to be 1 for (0,1) and (1,0), but 0 for (0,0) and (1,1). These positive examples occupy opposite corners of the square. No single straight line can separate both positive points from

both negative points. This is a limitation of the perceptron hypothesis class, not a poor choice of learning rate.

Learning rate	Maximum epochs	Final predictions	Converged?	Interpretation
0.1	30	[1, 1, 0, 0]	No	The update sequence does not reach a zero-error solution.
0.5	30	[1, 1, 0, 0]	No	The update sequence does not reach a zero-error solution.

Changing the learning rate changes the size of each weight and bias update. It can change the path and speed of learning, but it cannot create a nonlinear decision boundary. The inability to represent XOR remains at both tested learning rates. A multilayer perceptron with at least one nonlinear hidden layer is required.

Q3 – DEVELOP AND VERIFY BACKPROPAGATION FOR A MULTILAYER PERCEPTRON (6 MARKS)

3.1 Network and forward-pass equations

The network contains two input neurons, two sigmoid hidden neurons and one sigmoid output neuron. The output is interpreted as the probability that the XOR review flag is 1.

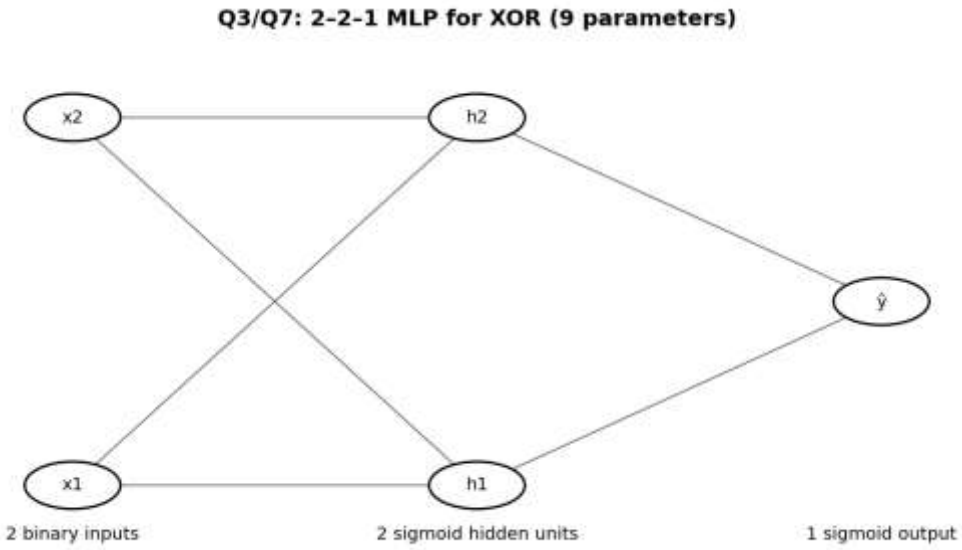


Figure 3. Two-input, two-hidden-unit, one-output multilayer perceptron.

$z_h = xW1 + b1; \quad h = \sigma(z_h)$

$z_o = hW2 + b2; \quad p = \sigma(z_o)$

$L = -[y \log(p) + (1-y) \log(1-p)]$

Sigmoid is $\sigma(a) = 1/(1+e^{(-a)})$ and its derivative is $\sigma(a)(1-\sigma(a))$. The hidden activation must be nonlinear. If every layer were linear, a composition of the layers would still be a linear transformation and could not represent XOR.

3.2 Backpropagation derivation

For sigmoid output combined with binary cross-entropy, the derivative of the loss with respect to the output pre-activation simplifies to $p - y$. For one example:

$\delta_o = \partial L/\partial z_o = p - y$

$\partial L/\partial W2 = h^T \delta_o; \quad \partial L/\partial b2 = \delta_o$

$\delta_h = (\delta_o\, W2^T) \odot h \odot (1-h)$

$\partial L/\partial W1 = x^T\, \delta_h; \quad \partial L/\partial b1 = \delta_h$

$W \leftarrow W - \eta\, \partial L/\partial W; \quad b \leftarrow b - \eta\, \partial L/\partial b$

For a mini-batch, the gradients are averaged across examples. The notebook implements the gradient equations directly and uses Adam for the XOR fit. Adam maintains first- and second-moment estimates of the gradients, applies bias correction, and scales each parameter update by the estimated gradient variance.

3.3 Manual numerical forward pass and one training step

Use x = [1,0], target y = 1, learning rate η = 0.10, W1 = [[0.1,0.2],[0.3,0.4]], b1 = [0.1,−0.1], W2 = [[0.2],[−0.3]] and b2 = [0.05].

Quantity	Calculated value
Hidden pre-activation z_h	[0.2, 0.1]
Hidden activation h	[0.549834, 0.524979]
Output pre-activation z_o	0.002473
Predicted probability p	0.500618
Binary cross-entropy loss	0.691911
Output delta δ_o	-0.499382
Hidden delta δ_h	[-0.024721, 0.03736]

The initial output is approximately 0.500618 and the loss is approximately 0.691911. The gradients and updated values after one gradient-descent step are:

W1 initial = [[0.100000, 0.200000], [0.300000, 0.400000]]

Gradient W1 = [[−0.024721, 0.037360], [0.000000, 0.000000]]

W1 updated = [[0.102472, 0.196264], [0.300000, 0.400000]]

b1 initial = [0.100000, −0.100000]; gradient b1 = [−0.024721, 0.037360]

b1 updated = [0.102472, −0.103736]

W2 initial = [[0.200000], [−0.300000]]; gradient W2 = [[−0.274577], [−0.262165]]

W2 updated = [[0.227458], [−0.273783]]

b2 initial = [0.050000]; gradient b2 = [−0.499382]; b2 updated = [0.099938]

3.4 Numerical gradient validation

A central finite-difference check perturbs W1[0,0] by ε = 0.00001 in the positive and negative directions and approximates the derivative as [L(θ+ε) − L(θ−ε)]/(2ε). The analytical and numerical gradients are:

Parameter	Analytical gradient	Finite-difference gradient	Absolute error
W1[0,0]	-0.0247210513	-0.0247210513	6.076e-12

The very small absolute difference supports the correctness of the implemented gradient for the checked parameter. It is a numerical verification of one selected component; additional parameters can be checked in the same way.

3.5 Training on all four XOR combinations

The 2–2–1 MLP was trained on all four rows using the custom NumPy Adam implementation with learning rate 0.05, seed 1 and 1020 update steps. The final binary cross-entropy is 0.001932.

x1	x2	Target	Final probability	Predicted class
0	0	0	0.002014	0
0	1	1	0.997767	1
1	0	1	0.998246	1
1	1	0	0.001721	0

The final predictions match all four XOR targets, giving 100% training accuracy on this four-row truth table. The decreasing loss curve shows that the optimizer reduced the binary cross-entropy during training.

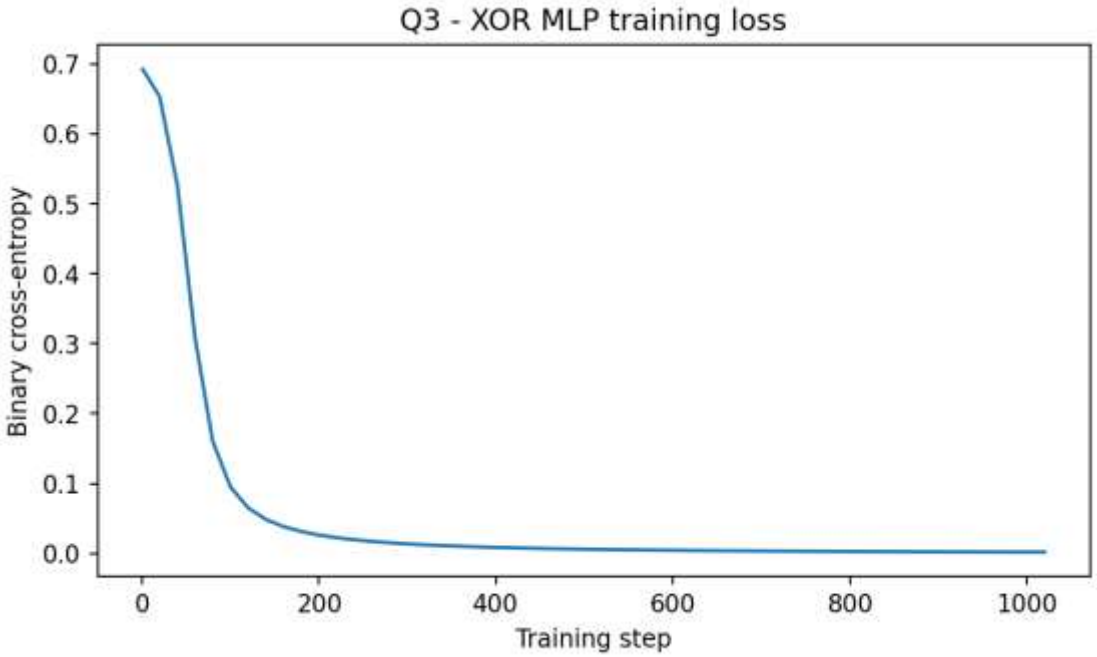


Figure 4. Binary cross-entropy during the NumPy 2–2–1 XOR training run.

Q4 – INVESTIGATE WHETHER A DEEPER NETWORK IS A BETTER DESIGN (6 MARKS)

4.1 Synthetic dataset and experimental protocol

A reproducible two-moons dataset was generated with 1,000 observations, two numerical features, noise = 0.25 and random_state = 42. The dataset contains two interleaving crescent-shaped classes, providing a nonlinear classification boundary as requested in the assignment brief. The stratified partition contains 600 training, 200 validation and 200 test observations.

Item	Setting
Dataset generator	sklearn.datasets.make_moons(n_samples=1000, noise=0.25, random_state=42)
Class counts	[500, 500]
Partitions	600 training / 200 validation / 200 test
Preprocessing	StandardScaler fitted on the 600 training rows only
Common optimizer	Adam
Learning rate	0.01
Batch size	32
Maximum epochs	100
Random seeds	11, 22 and 33
Checkpoint rule	Keep the epoch with lowest validation BCE loss

4.2 Candidate network structures and parameter counts

The shallow model has one hidden layer with 32 ReLU units. The deep model has three hidden layers with 16 ReLU units each. Both have two input features and one sigmoid output.

Shallow: $(2 \times 32 + 32) + (32 \times 1 + 1) = 96 + 33 = 129$ parameters

Deep: $(2 \times 16 + 16) + (16 \times 16 + 16) + (16 \times 16 + 16) + (16 \times 1 + 1) = 48 + 272 + 272 + 17 = 609$ parameters

Before training, the shallow model is expected to learn a useful nonlinear boundary with lower computation. The deep model has more representational capacity, but the extra depth may not help this relatively simple two-dimensional problem and could increase optimisation difficulty.

4.3 Results over three initialisation seeds

Architecture	Parameters	Mean validation loss	Mean validation accuracy	Mean training time (s)
One hidden layer, 32 units	129	0.1603	91.83% ± 0.29%	0.322
Three hidden layers, 16 units each	609	0.1706	92.50% ± 0.50%	0.805

The deep network has a slightly higher mean validation accuracy, but the shallow model has the lower mean validation loss (0.1603 compared with 0.1706), uses 129 rather than 609 parameters, and takes about half the measured training time. Validation loss was selected as the primary architecture-selection criterion because it evaluates the quality of predicted probabilities, not only whether they cross the 0.50 threshold.

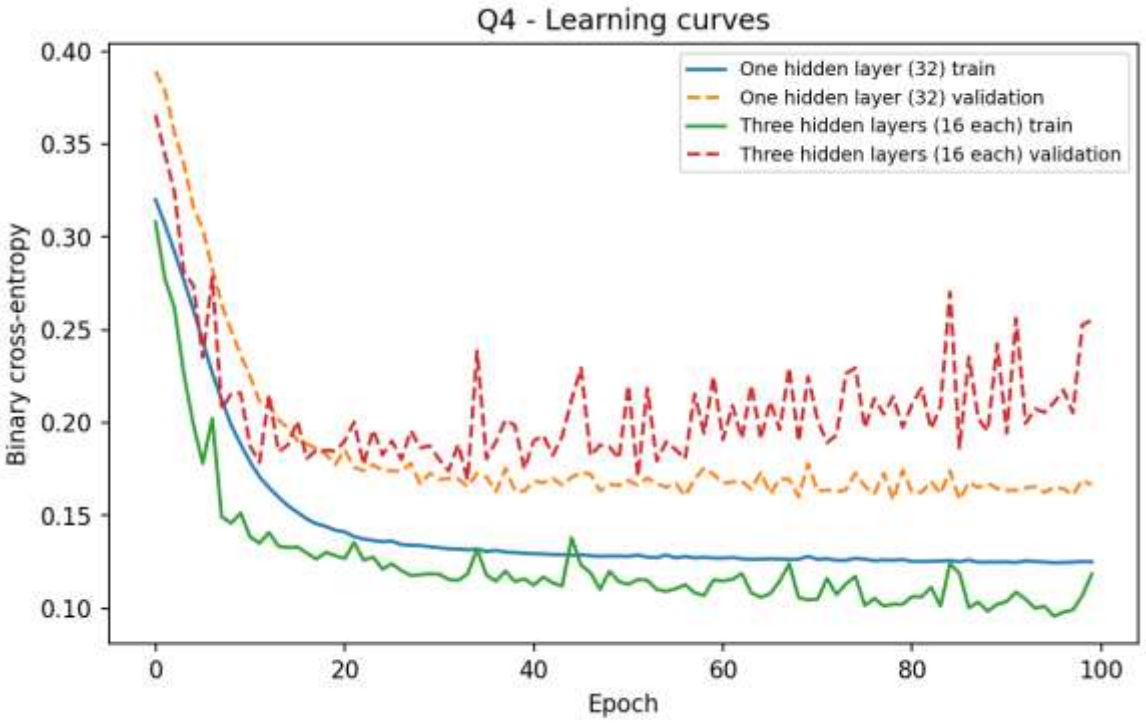


Figure 5. Training and validation loss for representative seed 11 runs.

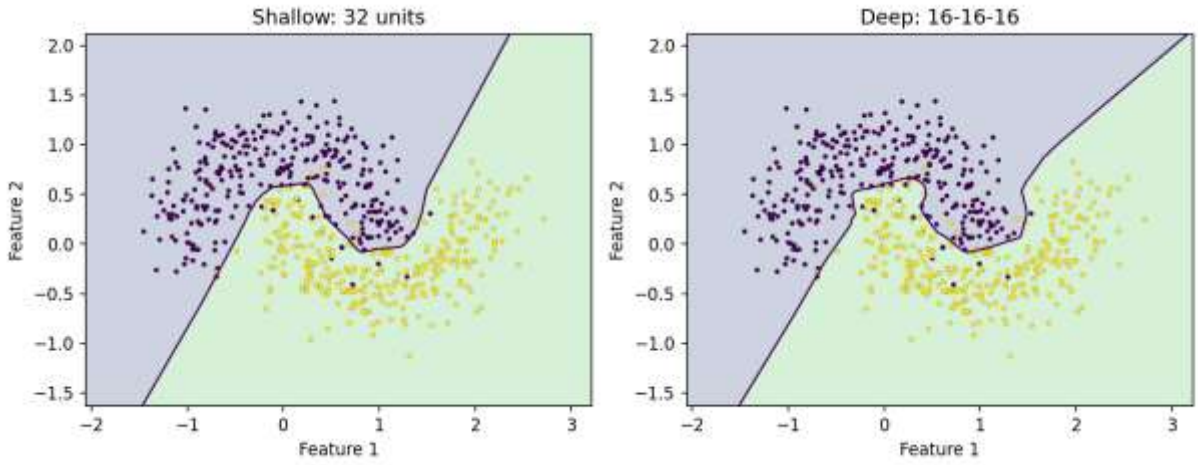


Figure 6. Learned decision boundaries for the shallow and deep networks (representative seed 11).

4.4 Validation-based recommendation and final test evaluation

The shallow network was selected using the mean validation loss across the three seeds. Only after that selection was fixed was the held-out test partition evaluated. Its test accuracies for seeds 11, 22 and 33 were [0.975, 0.975, 0.98], giving a mean test accuracy of 97.67% (standard deviation 0.29 percentage points).

The results do not support the claim that more depth is automatically better. The deep network achieved a modestly higher validation accuracy, but its validation loss was worse and its parameter count was about 4.72 times larger. The shallow network is therefore the more efficient choice under the stated validation-loss criterion. The experiment is limited to one synthetic dataset and three seeds; it does not prove that shallow networks are always preferable.

Q5 – DIAGNOSE OVERFITTING AND EVALUATE REGULARIZATION (7 MARKS)

5.1 Limited-data setup and baseline prediction

A reproducible subset of 100 labelled observations was sampled from the original 600-row training partition using seed 2026. The 200 validation and 200 test observations were retained as separate partitions. A new StandardScaler was fitted only on those 100 training rows. The proposed large network uses hidden widths 128–128–64 and has 25,217 trainable parameters.

Parameter count = (2×128+128) + (128×128+128) + (128×64+64) + (64×1+1) = 25,217

Before training, the expected pattern is that the large network may fit the 100 training examples more closely than it generalises to the validation examples. A widening gap between training and validation loss would indicate overfitting.

5.2 Experimental interventions

Method	Setting	Purpose
Unregularised baseline	No L2 penalty; no dropout; 140-epoch maximum	Reference behaviour of the large network.
L2 regularization	$\lambda = 0.01$	Penalise large weights and discourage unnecessarily complex fits.
Dropout	Dropout probability = 0.40 on hidden activations	Reduce reliance on particular hidden units during training.
Early stopping	Validation-loss patience = 20 epochs; restore best checkpoint	Stop once validation loss has failed to improve for 20 epochs.

All four experiments use the same 100 training examples, the same validation and test partitions, the same 128–128–64 architecture, Adam with learning rate 0.01 and batch size 16. The baseline, L2 and dropout runs use a 140-epoch maximum; early stopping ends after 49 epochs in this run and retains the best validation checkpoint.

5.3 Results and overfitting diagnosis

Method	Best epoch	Best val loss	Best val acc.	Final train acc.	Final val acc.	Final val loss
Baseline	29	0.2840	89.0%	94.0%	88.0%	0.5450
L2, λ=0.01	136	0.3083	92.0%	92.0%	91.5%	0.3216
Dropout 0.40	98	0.2431	91.5%	92.0%	90.0%	0.3651
Early stopping	29	0.2840	89.0%	94.0%	90.5%	0.4093

The baseline is overfitting: by the final epoch its training accuracy is 94% while validation accuracy is 88%; training loss is 0.0995 but validation loss has risen to 0.5450. The best validation loss occurred at epoch 29, but training continued to epoch 140 and validation loss deteriorated. This is the expected behaviour when a large model can adapt too closely to a small labelled sample.

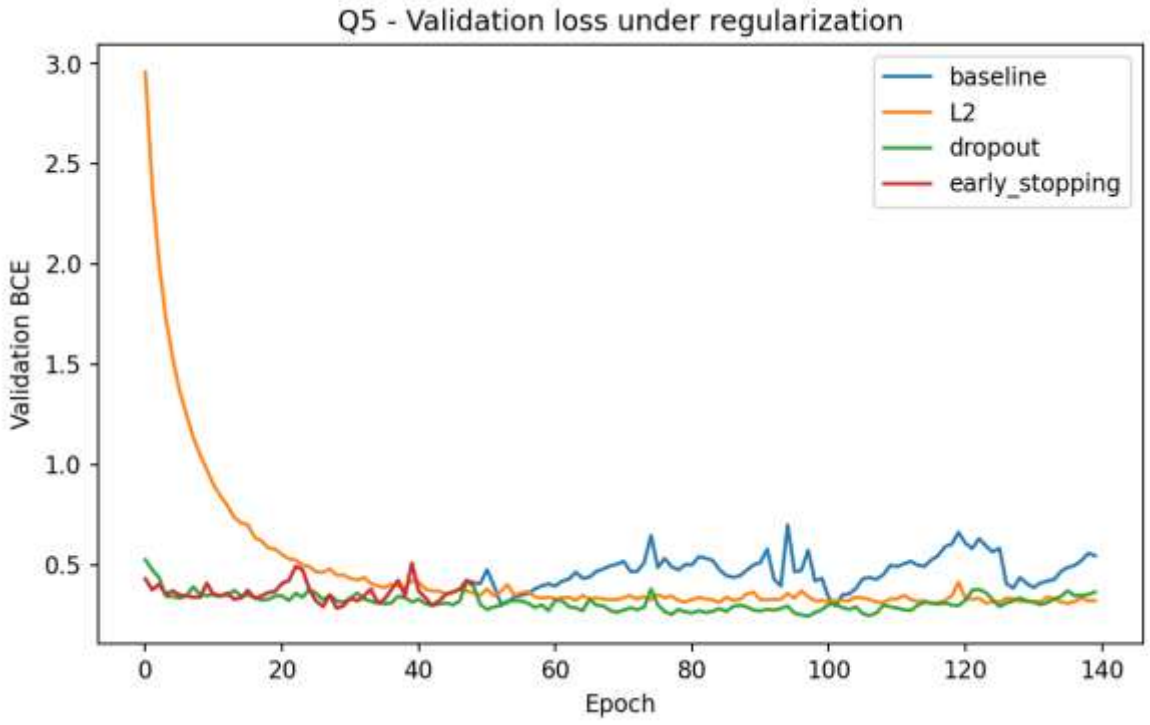


Figure 7. Validation loss curves for the baseline and regularization interventions.

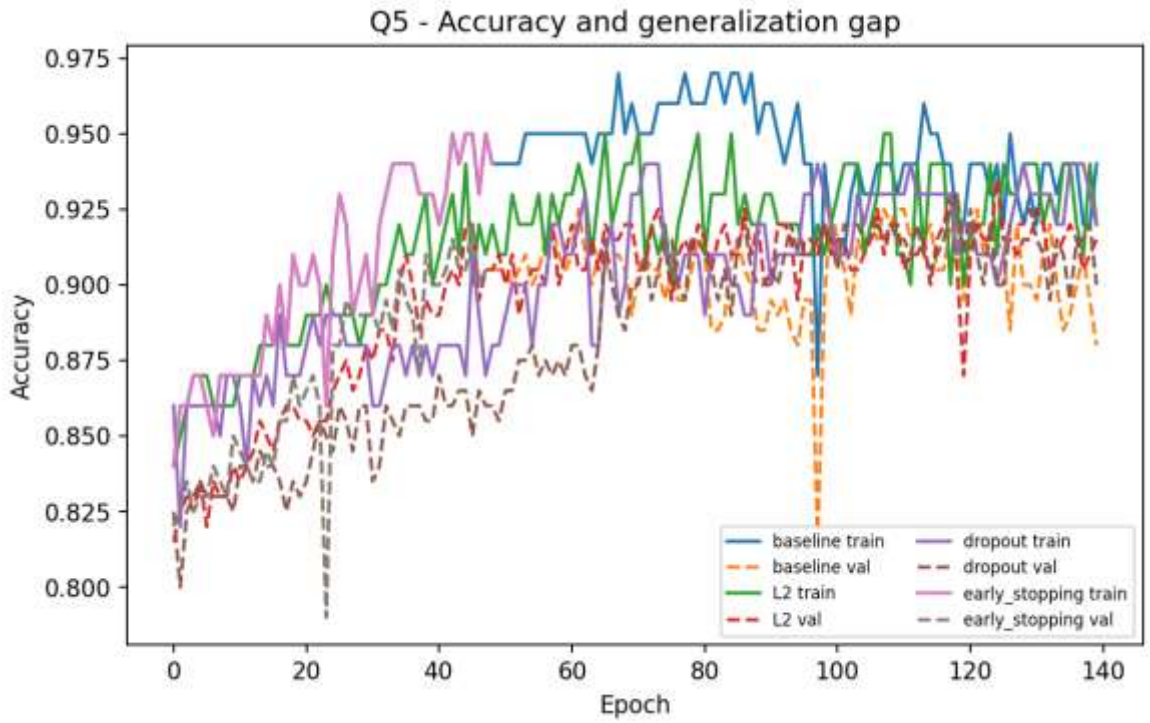


Figure 8. Training and validation accuracy across the limited-data experiments.

5.4 Interpretation and selected configuration

Dropout produced the lowest best validation loss (0.2431) and validation accuracy of 91.5%. L2 achieved the highest best validation accuracy (92.0%) and substantially reduced the final validation loss compared with the unregularised baseline. Because validation BCE was declared as the primary selection criterion before inspecting the test result, dropout is selected. After this choice, the held-out test accuracy of the selected dropout model is 93.0%.

Early stopping reduced training time from approximately 0.96 seconds for the baseline to approximately 0.34 seconds, a reduction of roughly 64%. It stopped at epoch 49 and retained epoch 29 as its best checkpoint. Early stopping alone did not improve the best validation checkpoint over the baseline in this particular run; its main benefit here was avoiding unnecessary later epochs.

Higher training accuracy does not necessarily mean a better model. The baseline reached 94% training accuracy but had a much worse final validation loss than the regularised models. The preferred model must be chosen with validation evidence, and the test set must be reserved for the final evaluation. The reported training times are environment-dependent and should be treated as approximate.

Q6 – ENGINEER A HYPERPARAMETER AND OPTIMIZATION STRATEGY (8 MARKS)

6.1 Search design and 12-run budget

The search investigates learning rate, batch size, hidden-layer width and optimizer strategy while using the same 600-row training and 200-row validation partitions. All candidates have an 80-epoch maximum, use training-only standardisation, and select their checkpoint by lowest validation BCE loss. The 12 runs are allocated to controlled comparisons rather than an unrestricted grid.

Run	Optimizer	Learning rate	Batch size	Hidden width	Seed / purpose
1	Adam	.001	32	32	61 – learning-rate sweep
2	Adam	.01	32	32	61 – learning-rate sweep
3	Adam	.05	32	32	61 – learning-rate sweep
4	Adam	.01	16	32	61 – batch-size sweep
5	Adam	.01	64	32	61 – batch-size sweep
6	Adam	.01	32	16	61 – width sweep
7	Adam	.01	32	64	61 – width sweep
8	SGD	.01	32	32	61 – optimizer comparison
9	SGD	.05	32	32	61 – optimizer comparison
10	SGD + momentum	.01	32	32	61 – momentum comparison

Run	Optimizer	Learning rate	Batch size	Hidden width	Seed / purpose
11	SGD + momentum	.05	32	32	61 – momentum comparison
12	Adam	.01	32	32	62 – repeat-seed check

6.2 Validation results for all candidate runs

Run	Best epoch	Best val loss	Val accuracy	Train accuracy at best	Time (s)
1	80	0.2644	88.0%	91.0%	0.230
2	50	0.1642	92.0%	95.0%	0.290
3	56	0.1469	92.0%	94.3%	0.255
4	72	0.1605	92.0%	95.5%	0.354
5	55	0.1637	92.5%	95.5%	0.168
6	80	0.1702	92.0%	95.0%	0.209
7	72	0.1591	92.0%	95.3%	0.220
8	80	0.3566	81.5%	86.3%	0.121
9	80	0.2471	90.0%	92.3%	0.136
10	80	0.1894	91.0%	95.2%	0.144
11	72	0.1657	92.5%	94.7%	0.156
12	80	0.1589	92.0%	95.0%	0.210

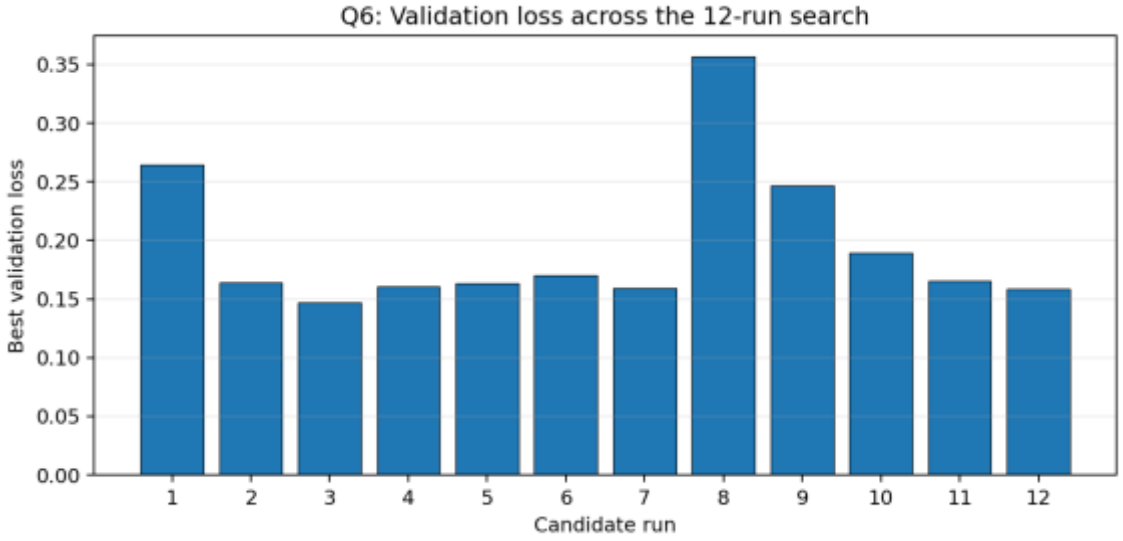


Figure 9. Best validation loss for each of the 12 candidate configurations.

6.3 Selection and optimizer interpretation

Run 3 was selected because it had the lowest validation loss (0.1469). Its configuration is Adam, learning rate 0.05, batch size 32, one hidden layer with 32 units and 129 trainable parameters. Its validation accuracy was 92.0% and the best validation checkpoint occurred at epoch 56.

The learning-rate sweep shows that 0.001 is too conservative under the 80-epoch budget: it produced validation loss 0.2644 and validation accuracy 88%. Increasing the Adam learning rate to 0.01 improved validation loss to 0.1642; 0.05 further reduced it to 0.1469 in this experiment. An excessively large learning rate can instead cause oscillation or divergence, so the observed result should not be extrapolated beyond the tested values.

Batch sizes 16 and 64 gave similar validation losses (0.1605 and 0.1637), while changing batch size also changes the number of parameter updates per epoch and wall-clock time. Width 16, 32 and 64 produced validation losses of 0.1702, 0.1642 and 0.1591 respectively; the improvement with width was small relative to the change in capacity. SGD at learning rate 0.01 performed poorly under the fixed budget, while SGD at 0.05 and SGD with momentum were more competitive. Adam converged to the lowest loss among the 12 candidates.

The repeat-seed run (run 12) achieved a validation loss of 0.1589, showing that initialisation can affect the result. Because there were only 12 candidates, conclusions about the globally optimal configuration are limited. A wider search or repeated seeds for each configuration would be required for a stronger claim.

After selecting run 3 using validation loss, the selected configuration was evaluated on the held-out test partition. Its final test accuracy was 97.0%. Test performance was not used to rank the 12 candidates.

Q7 – DIAGNOSE, REDESIGN AND VALIDATE A NEURAL NETWORK SOLUTION (13 MARKS)

7A. Diagnose the learning or computational bottleneck (2 marks)

The selected limitation is the single-layer perceptron failure on the XOR review flag in Q2. At both learning rates 0.10 and 0.50, the perceptron did not converge within 30 epochs. The final prediction vector was [1,1,0,0], which agrees with only two of the four XOR labels and gives 50% accuracy. The geometry in Figure 2 shows why: the positive XOR points are on opposite corners, so no single straight decision boundary can separate them.

The primary bottleneck is model representation, not data quantity or compute budget. The same four truth-table examples are fully specified, and a higher learning rate cannot make a linear classifier represent a nonlinear concept. The learning-rate experiment supports this diagnosis because neither setting reaches zero training error.

7B. Measurable redesign objective (2 marks)

Baseline: a two-input perceptron with two weights and one bias (three trainable parameters), with observed 50% accuracy on XOR after 30 epochs. Redesign objective: use a compact nonlinear model that correctly classifies all four rows of the supplied XOR truth table (100% training accuracy), while keeping the network small and the decision process understandable.

7C. Revised learning strategy (4 marks)

Replace the single output perceptron with a 2–2–1 multilayer perceptron. The two input values feed two sigmoid hidden neurons; both hidden neurons connect to one sigmoid output. The hidden nonlinearities allow the network to form intermediate feature combinations that a single linear threshold cannot express.

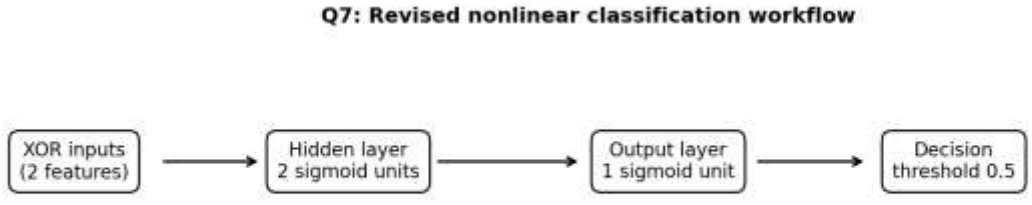


Figure 10. Revised nonlinear workflow for the XOR review flag.

$h = \sigma(xW1 + b1)$; $p = \sigma(hW2 + b2)$; class = 1 if $p \geq 0.50$

Parameters = $(2 \times 2 + 2) + (2 \times 1 + 1) = 6 + 3 = 9$

The redesigned network uses nine trainable parameters, three times as many as the perceptron, but remains a very small model. The implementation computes gradients using the chain rule and uses Adam to update parameters. The cost is a small increase in computation and a dependence on weight initialisation and training settings. The model is more capable, but a larger network is not automatically needed for this four-point problem.

7D. Prediction before implementing (1 mark)

Prediction: the perceptron will remain unable to separate XOR regardless of the tested learning rates, while the nonlinear 2–2–1 MLP should reduce BCE and classify all four truth-table rows correctly. The primary expected improvement is training accuracy from 50% to 100%, with final BCE approaching zero.

7E. Implementation and validation (2 marks)

The implementation reuses the custom NumPy 2–2–1 network from Q3 with the XOR training rows. It uses seed 1, Adam, learning rate 0.05 and a recorded 1,020 update steps. The final output probabilities and classifications are:

Input (x1,x2)	Target	Perceptron prediction	MLP probability	MLP prediction
(0,0)	0	1	0.002014	0
(0,1)	1	1	0.997767	1
(1,0)	1	0	0.998246	1
(1,1)	0	0	0.001721	0

Model	Parameters	Training accuracy	Final BCE	Outcome
Single-layer perceptron	3	50%	Not converged	Cannot represent XOR
2–2–1 sigmoid	9	100%	0.001932	All four labels

Model	Parameters	Training accuracy	Final BCE	Outcome
MLP				reproduced

7F. Final design justification and limitations (2 marks)

The redesign achieved the stated objective on the supplied XOR truth table: all four predictions are correct and the final BCE is approximately 0.001932. The improvement is explained by the addition of a nonlinear hidden layer, not by simply increasing the learning rate. The model remains small enough to implement and inspect, and the output threshold is explicit.

The main limitation is that four logical examples are not a realistic validation dataset. Perfect performance on the XOR truth table demonstrates representational capability, not generalisation to noisy real-world warehouse observations. A production review-flag system would require a larger labelled dataset, noise checks, validation, calibration, monitoring and a clear human override process. The original perceptron remains preferable for genuinely linearly separable rules such as the AND dispatch rule because it is simpler and needs fewer parameters.

OVERALL CONCLUSION

The assignment experiments show that neural-network design must match the structure of the target relationship. A single-layer perceptron learns the linearly separable AND dispatch decision but fails on XOR at both tested learning rates. A 2–2–1 sigmoid MLP learns XOR with 100% accuracy, and its manually derived gradients agree with a numerical finite-difference check.

For the synthetic nonlinear two-moons classification task, the shallow network with one 32-unit hidden layer achieved lower average validation loss than the three-hidden-layer alternative while using 129 rather than 609 parameters. The deep model had slightly higher validation accuracy, but the reduction in validation loss and additional parameters did not justify the larger model under the declared selection rule. The limited-data experiment showed a clear baseline overfitting pattern; L2 and dropout improved validation behaviour, while early stopping reduced training time by ending the run when validation loss stopped improving.

The 12-run search selected Adam with learning rate 0.05, batch size 32 and width 32 using validation loss. The final test accuracy of the selected configuration was 97.0%. The Day-4 redesign replaced the linear XOR perceptron with a compact nonlinear MLP and achieved the predefined 100% truth-table accuracy objective. The experiments are reproducible and useful for illustrating the assessed concepts, but results on synthetic data and tiny logical examples must not be presented as evidence of real manufacturing or warehouse performance.

REPRODUCIBILITY AND SUBMISSION NOTES

- The accompanying notebook contains all code for Q1–Q7, including explicit NumPy gradient calculations, training algorithms, the synthetic dataset generator, experiment loops and graph generation.
- The notebook creates an a2_outputs folder containing the generated 1,000-row dataset, result JSON, 12-run configuration table and figures.
- All random seeds and data partitions are recorded in the notebook. Training and validation results are used for model selection; held-out test results are reported only after selection.
- Fill in the cover-page student details and verify the final report against the notebook outputs before submission.